

ResumeForge: A Local-First, Grounding-Constrained System for LLM-Assisted Resume Tailoring

Kaustubha Venkata Eluri
Northeastern University

Abstract—Large language models are increasingly used to help job seekers tailor resumes to specific job descriptions, but this convenience introduces two under-examined risks: the model may fabricate experience, metrics, or job titles that the user never held, and the user’s private career history may be transmitted to a third-party chatbot with no data-handling guarantees. This paper documents ResumeForge, a local-first Next.js application that tailors a single, user-authored master resume against a job description while constraining the generator to facts that are actually present in that master resume. We describe the system’s local-first data model (a single JSON file, no cloud storage unless explicitly opted into), its dual large-language-model (LLM) backend design (a local Ollama model versus a cloud Groq model), its keyword-based Applicant Tracking System (ATS) match scoring, and its server-side LaTeX/PDF export pipeline built on the Tectonic typesetting engine. We describe the system’s grounding constraint – a system-prompt-level instruction restricting the generator to facts present in the master resume – as the project’s central design contribution, and report a real, documented quality-assurance finding: a LaTeX-escaping defect in the export pipeline caused by a naive two-pass regex approach, and its fix via paragraph-first escaping. We close with the system’s limitations and directions for future evaluation.

Index Terms—resume generation, large language models, hallucination prevention, local-first software, applicant tracking systems, LaTeX document generation

I. INTRODUCTION

Resume tailoring – rewriting a resume’s emphasis, wording, and keyword coverage to match a specific job posting – is a well-established practice for improving the odds that a resume passes both automated Applicant Tracking Systems (ATS) and human screeners. Large language models are a natural fit for this task: they can rephrase and reorganize a candidate’s existing experience quickly. In practice, however, two problems commonly arise when a job seeker uses a general-purpose chatbot for this task.

First, an LLM asked to “tailor” a resume to a job description has an incentive, implicit in its training objective, to produce output that reads as an excellent match for the posting. Without an explicit constraint, this can manifest as fabricated metrics, invented job titles, or skills the candidate does not actually possess – a serious problem for a document whose entire purpose is a truthful, verifiable account of the candidate’s history.

Second, pasting a resume and a job description into a generic cloud chatbot means transmitting personally identi-

fying career history and, often, an employer’s proprietary job posting text to a third party with no accounting of how that data is stored, retained, or used to train future models.

ResumeForge is a project that addresses both concerns directly, by construction rather than by policy. It is a local-first Next.js application in which (1) all resume and history data is stored in a single JSON file on the user’s own machine unless the user explicitly opts into a cloud LLM backend, and (2) the generation step is constrained, at the system-prompt level, to use only facts present in a user-authored “master resume,” with no invented metrics or job titles. The system additionally provides a keyword-coverage ATS score, and exports the tailored resume as PDF, LaTeX, or plain text, with PDF compilation performed server-side via the Tectonic LaTeX engine.

This paper documents the system’s architecture, its grounding-constraint design, and its export pipeline, and reports the concrete quality-assurance issues discovered and fixed during development, as recorded in the project’s own documentation. Consistent with the scope of that documentation, this paper does not claim any user-adoption or resume-acceptance-rate figures; no such data was collected or is claimed by the project.

II. RELATED WORK

A. LLM-Based Document Generation

LLMs are widely used to draft and rewrite structured text documents such as cover letters, resumes, and reports. The generic pattern – prompt the model with a source document and a target context, ask it to produce a rewritten version – is effective at producing fluent output, but conflates two distinct sub-tasks: rephrasing/reorganizing existing content, and inferring or inventing content the model believes would be a good fit. Systems that do not distinguish these sub-tasks in their prompting strategy are prone to the latter behavior whenever the source document under-specifies what the target context calls for.

B. Grounding and Hallucination Prevention

A general strategy for reducing fabrication in LLM output is to explicitly ground generation in a bounded, user-supplied source of truth, and to instruct the model, at the system-prompt level, not to introduce facts absent from that source. This is a lighter-weight alternative to retrieval-augmented

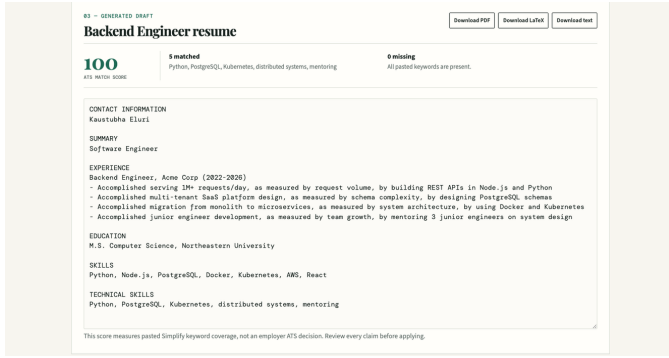


Fig. 1. The generated-draft view of ResumeForge, showing a tailored resume produced from the user’s master resume and a target job description, together with the computed ATS keyword-coverage score.

generation pipelines or post-hoc fact-checking passes: rather than verifying output after the fact, the constraint is stated as an instruction at generation time, and the source of truth is small enough (a single resume) to be included in full in the prompt context. ResumeForge’s design follows this pattern, applying it to a domain – resume tailoring – where fabrication has direct real-world consequences for the user.

C. ATS Keyword Matching

Keyword-matching heuristics are a common, low-cost proxy for how an automated screening system might assess a resume against a job posting: keywords are extracted from the job description and checked for presence or absence in the candidate document. Such heuristics are widely understood to be an approximation, not a guarantee, of how any specific employer’s actual ATS software will score a submission, since real ATS implementations are proprietary and vary in their matching logic (synonym handling, weighting, section parsing, etc.). ResumeForge’s scoring feature is explicitly scoped in its own documentation as a keyword-coverage signal rather than a claim about real-world ATS outcomes.

III. SYSTEM ARCHITECTURE

ResumeForge is built on Next.js 15 with React 19 and TypeScript. The system’s structure is summarized textually below, per the project’s own architecture diagram. Figure 1 shows the application’s generated-draft view, which presents the tailored resume text alongside its computed ATS keyword-coverage score, illustrating the primary end-user-facing surface of the tailoring feature described in this section.

A. Local-First Data Model

All application state – the user’s master resume and the full history of every generated, tailored version – is persisted to a single file, `data/resumeforge.json`, on the user’s own machine. No cloud storage is used at any point in the default configuration. The application’s own documentation states this explicitly: the master resume is “saved locally, never sent to a cloud AI service unless you opt into Groq.” This is a deliberate architectural choice, not merely a default setting:

the only way resume content leaves the machine is if the user sets a `GROQ_API_KEY` environment variable to enable the cloud backend.

B. Application Layer

The Next.js frontend (`app/page.tsx`) presents a master-resume editor, a tailoring form (job description and target keywords), a generated-draft view with ATS score, and a history list. It communicates via `fetch` with a set of API routes:

- `/api/master` – read/write the master resume;
- `/api/tailor` – generate a tailored draft and compute its ATS score;
- `/api/pdf` – invoke the Tectonic compiler and stream back a PDF;
- `/api/applications` – list the saved history of previously generated versions.

C. Dual LLM Backend

The `/api/tailor` route dispatches to one of two generation functions: `generateWithOllama()`, which calls a local Ollama server at `localhost:11434` using the `qwen2.5:7b` model by default (overridable via `OLLAMA_MODEL`, e.g. to the lighter `qwen2.5:3b` on memory-constrained machines), or `generateWithGroq()`, which calls the cloud-hosted `llama-3.3-70b-versatile` model on `api.groq.com`. The backend is selected automatically based on the presence of a `GROQ_API_KEY` environment variable: if it is set, the cloud backend is used and resume/job-description content is sent to Groq’s API; if it is unset, generation is fully local. This design lets a user trade privacy for latency (the documentation reports local generation as taking “a minute-plus” on constrained hardware versus approximately three seconds for the cloud backend) without changing any other part of the application.

D. History

Every generated tailored version is saved with its associated company, role, ATS score, and timestamp. Selecting a history entry restores not just the generated output but also the original form inputs that produced it, allowing the user to revisit or regenerate a prior tailoring session.

IV. GROUNDING CONSTRAINT DESIGN

The central design contribution of ResumeForge is not the use of an LLM to rewrite a resume – that pattern is common – but the explicit, system-prompt-level constraint placed on that rewriting: the generator is instructed to draw only from facts present in the user’s master resume, and is directed not to invent metrics or job titles that do not appear there.

This constraint matters specifically because of the failure mode it targets. A resume-tailoring prompt, by its nature, asks the model to make the candidate look like a strong match for a specific job posting. Left unconstrained, an LLM has every incentive, given its training to produce helpful and contextually

appropriate text, to *add* content that would make the match look stronger – a quantified metric that was never measured, a title that sounds more senior than the one actually held, a skill mentioned in the job description but never used by the candidate. Because a resume’s entire function is to be a truthful account of the candidate’s history, this specific failure mode is far more consequential here than in many other LLM writing tasks: a fabricated bullet point is not merely inaccurate prose, it is a claim the candidate may be asked to defend in an interview, or that may constitute deception to a prospective employer.

ResumeForge’s approach is to state the constraint directly as an instruction to the model at generation time, coupled with providing the master resume in full as the only source of factual content in the prompt context. This is architecturally simple – it requires no retrieval system, no post-generation fact-checking pass, and no fine-tuning – and it keeps the constraint co-located with the one artifact (the master resume) that defines what is true for a given user. The tradeoff, discussed further in Section VII, is that this is an instruction-following constraint enforced by the LLM itself rather than a constraint verified by any external mechanism; the system does not currently perform automated post-hoc verification that generated output stays within the bounds of the master resume.

V. PDF/LATEX EXPORT PIPELINE

ResumeForge exports tailored resumes in three formats: PDF, raw LaTeX (`.tex`), and plain text.

A. Server-Side Compilation via Tectonic

PDF export is performed server-side, in the `/api/pdf` route, using Tectonic, a self-contained LaTeX engine that requires no separate system-wide LaTeX distribution. The route writes intermediate files to a temporary directory, invokes the Tectonic binary as a subprocess, and streams the resulting PDF back to the client; the temporary directory is cleaned up in a `finally` block regardless of whether compilation succeeds or fails. Performing compilation server-side, rather than via a client-side LaTeX/WASM toolchain, keeps the browser JavaScript bundle free of that dependency.

B. Paragraph-Aware Escaping

Converting arbitrary user- and LLM-generated text into valid LaTeX source requires escaping special characters (`#`, `%`, `&`, `_`) and converting blank-line paragraph breaks into LaTeX line breaks. The module `lib/latex.ts` implements this by first splitting the input text on blank lines into paragraphs, and only then joining the lines within each paragraph with `\\`. This paragraph-first ordering is a deliberate correction of a naive two-pass regex-replace approach, which the project’s documentation identifies as producing invalid `\\` sequences whenever the input text contains a paragraph break, because the naive approach would double-process an inserted `\par` newline. This is discussed further as a concrete QA finding in Section VI.

VI. EVALUATION

Consistent with the project’s own documentation, this paper reports only the quality-assurance evidence actually recorded for the system; no user study, adoption metrics, or resume-acceptance-rate figures were collected, and none are claimed here.

A. Documented Defect: LaTeX Escaping

During development, a defect was identified in the LaTeX export path: an initial two-pass regex-based escaping implementation produced invalid LaTeX output whenever generated resume text contained a paragraph break, because the two passes interacted to double-process inserted paragraph-break markers into malformed line-break sequences. This was found and fixed by restructuring `lib/latex.ts` to split the input into paragraphs *before* performing line-joining, rather than attempting both transformations in sequential passes over the whole text. This fix is documented as one of the project’s explicit “design decisions worth knowing,” and represents a concrete, verifiable instance of a defect being found and corrected in the export pipeline as a direct consequence of exercising the PDF export feature against real generated resume text (i.e., multi-paragraph output containing blank-line breaks).

B. Design-Level Robustness Choices

Two further design choices function as evaluation-informed hardening rather than newly reported bugs, but are documented as deliberate responses to failure conditions encountered in the export pipeline: (1) performing PDF compilation server-side via a subprocess invocation of Tectonic, with temporary-directory cleanup executed in a `finally` block so that cleanup occurs regardless of whether compilation succeeds or throws; and (2) restoring the full set of tailoring-form inputs (not merely the generated output text) when a user selects a prior entry from history, so that a previously generated version can be inspected or regenerated in its original context.

VII. LIMITATIONS

Several limitations follow directly from the system’s documented design:

- **Instruction-following, not verified, grounding.** The constraint that the generator use only facts from the master resume is enforced by prompting the LLM to follow that instruction; the system does not perform automated, independent verification (e.g., a separate fact-checking pass) that a given generated draft actually stays within the bounds of the master resume. The strength of the grounding guarantee is therefore bounded by the underlying model’s instruction-following reliability.
- **Keyword-coverage scoring is an approximation.** The ATS score is explicitly scoped by the project as a keyword-coverage signal, not a prediction of how any specific employer’s actual, proprietary ATS software will score the resume.

- **Local generation performance.** On memory-constrained hardware (the documentation cites an 8 GB MacBook Air), local generation via the default `qwen2.5:7b` Ollama model can take over a minute, which may push users toward the faster but non-private Groq cloud backend.
- **Cloud backend forfeits the privacy guarantee.** When `GROQ_API_KEY` is set, resume and job-description content is sent to Groq’s API; the system’s privacy property holds only for the default, local configuration.
- **No externally validated evaluation.** As documented, the only concrete evaluation evidence available is the LaTeX-escaping defect described in Section VI; there is no controlled study of resume quality, ATS-score accuracy, or grounding-constraint adherence across a broader corpus of resumes and job descriptions.

VIII. CONCLUSION AND FUTURE WORK

ResumeForge demonstrates a practical, lightweight pattern for reducing a specific and consequential failure mode of LLM-based document generation – fabrication of unverifiable facts – by constraining generation, at the prompt level, to a single user-authored source of truth, combined with a local-first data architecture that keeps that source of truth off the network by default. Its dual-backend design (local Ollama versus cloud Groq) makes the privacy/latency tradeoff explicit and user-controlled rather than fixed by the application. Its export pipeline demonstrates that a self-contained LaTeX engine (Tectonic) can be used server-side to avoid both a system-wide LaTeX dependency and a client-side WASM toolchain, and its documented LaTeX-escaping fix illustrates a concrete instance of the paragraph-structure edge cases that arise when converting arbitrary generated text to LaTeX.

Future work suggested directly by the system’s documented limitations includes: an automated post-generation verification pass that checks a generated draft’s claims against the master resume, to strengthen the grounding guarantee beyond prompt-level instruction-following; a controlled evaluation of ATS keyword-score correlation with outcomes from real ATS software; and a systematic before/after comparison of local Ollama model options (e.g., `qwen2.5:3b` versus `qwen2.5:7b`) on generation quality versus latency for memory-constrained hardware.

REFERENCES

- [1] IEEE, “IEEE Conference Proceedings Formatting Guidelines,” IEEE, 2023.